

Clase teórica 5 : Ruteo

Tomás F. Melli

September 2025

Índice

1	Introducción	2
1.1	Algoritmos y Protocolos	2
1.2	Escalabilidad	2
1.3	Sistemas Autónomos (AS)	2
1.4	Reenvío vs. Ruteo	3
1.5	Ruteo en el modelo OSI	4
2	Protocolos de Ruteo Interno	5
2.1	RIP - Routing Information Protocol	5
2.1.1	Características principales	6
2.2	Distance Vector	6
2.2.1	Ejemplo I	7
2.2.2	Convergencia Global en Vector de Distancia	8
2.2.3	Ejemplo II	8
2.2.4	Ciclos de Actualización	9
2.2.5	Heurísticas para Romper Ciclos en Vector de Distancia	11
2.3	Formato paquete RIPv2	13
2.4	RIP como un "daemon"	14
2.5	Algoritmo de Estado del Enlace - Link State	14
2.5.1	Principios Fundamentales	14
2.5.2	Pasos del Algoritmo de Estado de Enlace	15
2.5.3	Paquete de Estado del Enlace (LSP)	15
2.5.4	Inundación Confiable (Reliable Flooding)	15
2.5.5	Cálculo de la Ruta Más Corta (Shortest Path Routing)	15
2.5.6	Uso del Algoritmo de Dijkstra en Ruteo	15
2.6	Comparación entre Algoritmos de Ruteo: Vector de Distancia vs Estado de Enlaces	17
2.7	Métricas de Enlace en ARPANET	17
2.8	OSPF (Open Shortest Path First)	18
2.8.1	Características Principales	18
2.8.2	OSPF Jerárquico	18
2.8.3	Tipos de Mensajes en OSPF	19
2.8.4	Mecanismo de Convergencia	19
2.8.5	Modificación del IPv4 Header al usar OSPF	19
2.8.6	Formato de Header OSPF	20
2.8.7	OSPF Link State Advertisement (LSA)	20
3	Ruteo Interdominio	21
3.1	EGP: Exterior Gateway Protocol (antecesor)	21
3.2	BGP: Exterior Gateway Protocol Moderno	21
3.2.1	Tipos de Sistemas Autónomos en BGP	21
3.2.2	Routers y Portavoces BGP	21
3.3	Network Access Points (NAPs)	22

1 Introducción

El ruteo es un componente esencial de las redes de datos, responsable de determinar los caminos que los paquetes deben seguir para alcanzar su destino. Dentro de este proceso, se distingue entre **ruteo interno** y **ruteo externo**, de acuerdo con el ámbito de aplicación de los protocolos.

- **Ruteo Interno (IGP, Interior Gateway Protocols)**: opera dentro de un **Sistema Autónomo (AS)**, por lo que se lo considera un protocolo **intradominio**. Ejemplos comunes incluyen **RIP (Routing Information Protocol)** y **OSPF (Open Shortest Path First)**.
- **Ruteo Externo (EGP, Exterior Gateway Protocols)**: conecta diferentes Sistemas Autónomos, gestionando el intercambio de rutas en el nivel **interdominio**. El principal protocolo de este tipo es el **BGP (Border Gateway Protocol)**.

La clave del diseño radica en la **autonomía y heterogeneidad**: cada AS administra sus tecnologías y políticas internas, manteniendo ocultos los detalles de su operación al resto de la red global.

1.1 Algoritmos y Protocolos

El ruteo puede modelarse como un problema de **grafos ponderados**, donde los nodos representan routers y los arcos representan enlaces con costos asociados. El objetivo es **encontrar el camino de menor costo entre dos nodos**, usando algoritmos eficientes y distribuidos.

Existen dos enfoques principales:

- **Ruteo Estático**: las rutas se configuran manualmente por el administrador. Tiene bajo costo de control, pero carece de adaptabilidad frente a fallas o cambios topológicos.
- **Ruteo Dinámico**: los routers actualizan sus tablas de forma autónoma según el estado de los enlaces, la carga de nodos o la detección de fallos. Utiliza algoritmos como:
 - **Distance Vector**: cada router intercambia periódicamente su tabla de ruteo con los vecinos. Ejemplo: **RIP**.
 - **Link State**: cada router descubre el estado de la red y calcula rutas con algoritmos como Dijkstra. Ejemplo: **OSPF**.
 - **Path Vector**: se utiliza en interdominios para mantener información de caminos completos. Ejemplo: **BGP**.

En el modelo OSI, el ruteo se sitúa en la **capa de red (Layer 3)**, construyendo y actualizando las tablas de rutas, mientras que el **reenvío (forwarding)** ocurre en base a dichas tablas, seleccionando la salida adecuada para cada paquete.

1.2 Escalabilidad

La **escalabilidad** es un desafío central en el diseño de protocolos de ruteo:

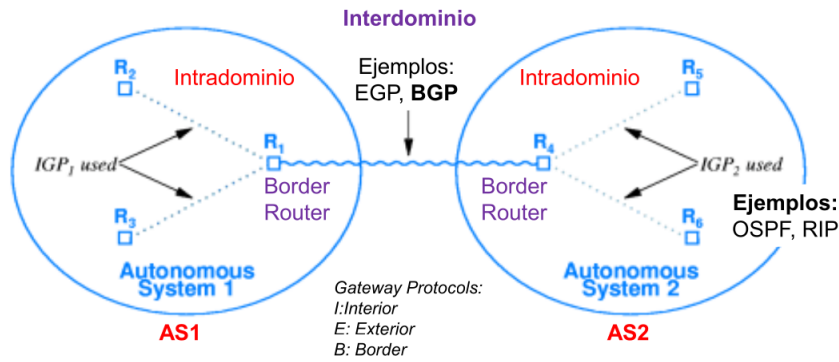
- **A nivel intradominio**: los protocolos IGP deben ser lo suficientemente eficientes para soportar miles de routers dentro de un AS, manteniendo baja sobrecarga de control y convergencia rápida.
- **A nivel interdominio**: Internet se compone de miles de AS, cada uno con sus políticas de enrutamiento. BGP debe manejar millones de rutas globales y garantizar estabilidad en un entorno altamente heterogéneo.

La solución a este problema se basa en la **jerarquía** y la **agregación de rutas**:

- Dentro de un AS, los protocolos como OSPF dividen la red en **áreas** para reducir la complejidad.
- Entre AS, BGP permite la **agregación de prefijos**, evitando la propagación innecesaria de rutas específicas.

1.3 Sistemas Autónomos (AS)

Un **Sistema Autónomo (AS)** es una red o conjunto de redes bajo el control de una única entidad administrativa, que comparte políticas de ruteo comunes. Internet puede visualizarse como un **grafo de AS**, cada uno conectado por routers de borde (*border routers*).



- **Intradominio:** protocolos internos como RIP u OSPF, ocultando detalles al exterior.
- **Interdominio:** protocolos externos como BGP, que gestionan políticas y acuerdos de peering entre AS.

La idea central es que **cada AS es autónomo** en su implementación tecnológica y en las políticas de ruteo, lo que permite la coexistencia de tecnologías heterogéneas (distintos protocolos, topologías y criterios de optimización).

1.4 Reenvío vs. Ruteo

Es fundamental distinguir entre:

- **Reenvío (Forwarding):** acción local de un router que consulta su tabla de forwarding y decide por qué interfaz enviar un paquete.
- **Ruteo (Routing):** proceso global y distribuido mediante el cual se construyen y actualizan las tablas de rutas.

(a)			lógica, modificable
	Prefix/Length	Next Hop	
Destino Layer3	18/8	171.69.245.10	Ruta Layer3
(b)			física, fija
	Prefix/Length	Interface	
Destino Layer3	18/8	if0	Destino Layer2
		MAC Address	8:0:2b:e4:b:1:2

(a) Tabla de **routing**
(b) Tabla de **forwarding**

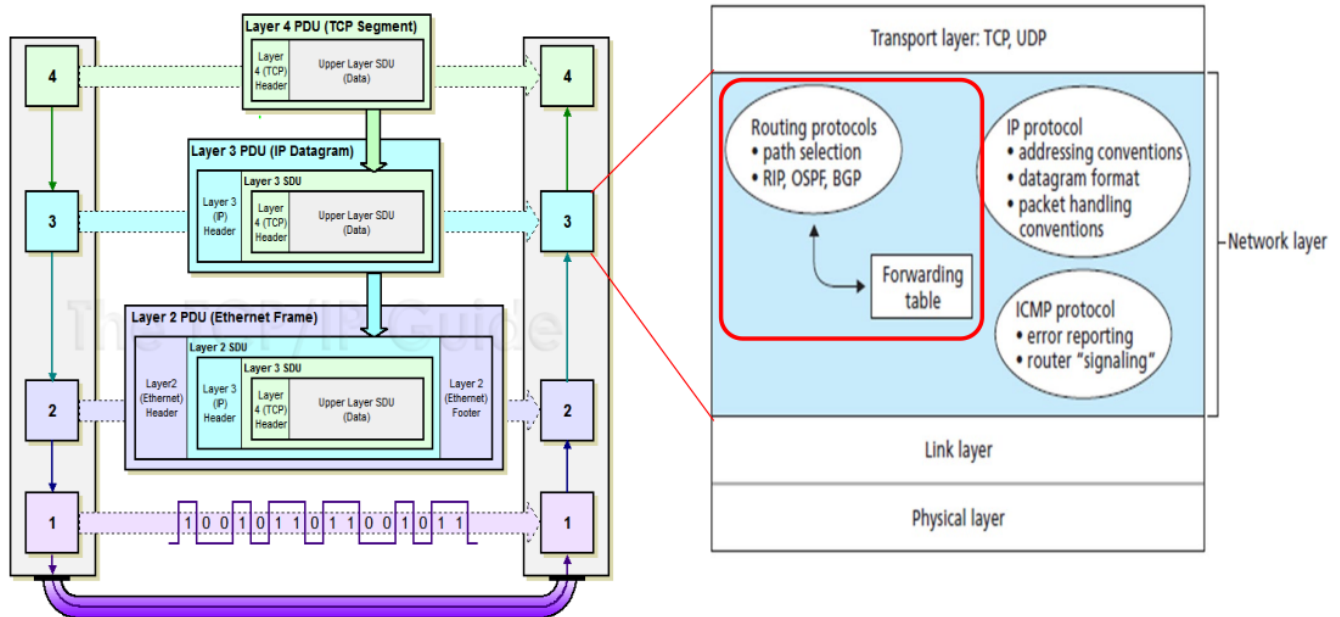
La imagen muestra la diferencia entre la **tabla de routing** y la **tabla de forwarding**:

- **Tabla de routing (a):**
 - Almacena la información de **ruta lógica** en la **capa de red (Layer 3)**.
 - Ejemplo: el prefijo 18/8 se alcanza a través del *next hop* 171.69.245.10.
 - Esta información es **lógica y modificable**, ya que depende de los protocolos de ruteo (RIP, OSPF, BGP).
- **Tabla de forwarding (b):**
 - Transforma la ruta lógica en información de **capa de enlace (Layer 2)**.
 - Para el mismo prefijo 18/8, se especifica la **interfaz de salida** (if0) y la **dirección MAC** de destino (8:0:2b:e4:b:1:2).
 - Esta información es **física y fija**, ya que corresponde al envío real del paquete en la red local (ejemplo: Ethernet).

En conclusión:

- **Routing (Layer 3):** decide el *camino lógico* que seguirá el paquete (qué salto siguiente usar).
- **Forwarding (Layer 2):** determina cómo **entregar físicamente** el paquete a través de una interfaz usando la dirección MAC.

1.5 Ruteo en el modelo OSI



La imagen muestra el proceso de **encapsulamiento** y el papel de la **capa de red** en el modelo OSI:

- **Encapsulamiento por capas** (lado izquierdo):
 - **Layer 4 (Transporte)**: segmento TCP o UDP.
 - **Layer 3 (Red)**: datagrama IP que contiene el segmento.
 - **Layer 2 (Enlace)**: trama Ethernet que encapsula el datagrama.
 - **Layer 1 (Física)**: bits transmitidos en el medio físico.
- **Funciones de la capa de red** (lado derecho):
 - Los **protocolos de ruteo** (RIP, OSPF, BGP) construyen la **tabla de ruteo**, que alimenta la **tabla de forwarding**.
 - Otros protocolos de red:
 - * **IP**: direccionamiento, formato de datagrama, convenciones de manejo.
 - * **ICMP**: reporte de errores y señalización de routers.

De esta forma, la **capa de red** es responsable de mover los paquetes de un origen a un destino, independientemente de la tecnología física subyacente.

Encapsulamiento en Redes

El **encapsulamiento** es el proceso mediante el cual cada capa del modelo OSI o TCP/IP toma los datos provenientes de la capa superior y les agrega información adicional (encabezados, y en algunos casos tráileres) necesaria para cumplir con sus funciones.

Proceso de Encapsulamiento

- **Capa de aplicación**:
 - Genera los datos de usuario, por ejemplo, un mensaje HTTP.
- **Capa de transporte**:
 - Encapsula los datos en un **segmento** (TCP) o **datagrama** (UDP).
 - Agrega información como puertos de origen y destino para identificar las aplicaciones extremas.

- **Capa de red:**
 - Encapsula el segmento en un **datagrama IP**.
 - Incluye direcciones IP de origen y destino, necesarias para el ruteo entre redes.
- **Capa de enlace:**
 - Encapsula el datagrama en una **trama Ethernet**.
 - Agrega direcciones físicas (MAC) y mecanismos de control de errores.
- **Capa física:**
 - Transmite la trama como una secuencia de **bits** a través del medio físico (cable, fibra, aire).

Desencapsulamiento

En el receptor, el proceso ocurre en sentido inverso: cada capa **elimina** su encabezado y tráiler, entregando los datos originales a la capa superior, hasta que llegan a la aplicación de destino.

2 Protocolos de Ruteo Interno

Los **protocolos de ruteo interno** (Interior Gateway Protocols, IGP) se utilizan dentro de un sistema autónomo para determinar las mejores rutas hacia las diferentes redes. Los dos más conocidos y utilizados en Internet son **RIP** y **OSPF**, los cuales difieren tanto en el tipo de algoritmo como en la forma de intercambiar información.

Cada Router	DISTANCE-VECTOR	LINK-STATE
¿Qué informa?	• Toda su Tabla de Ruteo	• Solo el Estado de sus Enlaces directos
¿A quién le pasa información?	• Solo a sus vecinos	• A toda la red (inundación)
Algoritmo utilizado	• Bellman-Ford Distribuido	• Dijkstra
Datos utilizados	• Información de los vecinos	• Estado de Enlaces de cada nodo
Estructuras de Datos	• Tabla de Distancias • Tabla de Ruteo	• Tabla de Estado de Enlaces • Tabla de Ruteo
Características	• Ciclos de Ruteo Gran variedad de Algoritmos: • Merlin-Segall • Jaffe-Moss • Esquema OP • Diffusing Comp • Cheng • Cálculo Distribuido	• Visión Consistente de la Red • Gran uso de CPU y Memoria • Algoritmo Básico único • Cálculo Centralizado
Ejemplo de Protocolos de Internet	Routing Information Protocol - RIP	Open Shortest Path First - OSPF

2.1 RIP - Routing Information Protocol

El **Routing Information Protocol (RIP)** es uno de los primeros **protocolos de ruteo interno (IGP)** utilizados en redes IP. Se basa en el enfoque de **vector de distancia**, lo que significa que cada router mantiene una tabla con la distancia (en número de saltos o *hops*) hacia cada destino conocido.

2.1.1 Características principales

- **Algoritmo:** utiliza el algoritmo de *Bellman-Ford distribuido*.
- **Métrica:** el costo de una ruta se mide en función de la cantidad de saltos (*hop count*).
 - Cada router intermedio cuenta como un salto.
 - El máximo número de saltos permitido es 15; un destino con 16 saltos es considerado inalcanzable.
- **Difusión periódica:** cada 30 segundos, un router envía toda su tabla de ruteo a sus vecinos.
- **Simplicidad:** fácil de implementar y con bajo consumo de CPU y memoria.
- **Limitaciones:**
 - Escasa escalabilidad debido a la restricción de 15 saltos.
 - Convergencia lenta en redes grandes o ante fallas.
 - Vulnerable a problemas de bucles de ruteo. Para mitigarlos, implementa técnicas como:
 - * *Split horizon*.
 - * *Route poisoning*.
 - * *Hold-down timers*.

2.2 Distance Vector

El algoritmo de **Vector de Distancia** es un método utilizado por los routers para calcular las mejores rutas hacia todos los nodos de una red. Sus características principales son:

- **Tabla de enrutamiento:** Cada nodo mantiene una tabla que contiene, para cada destino de la red, los siguientes campos:

(Destino, Costo, NextHop)

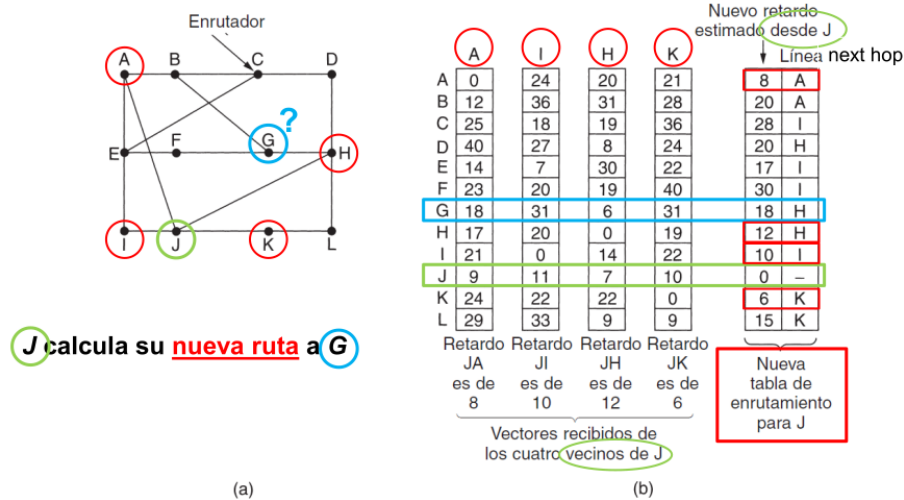
- **Intercambio de información:** Los nodos intercambian actualizaciones solo con sus vecinos directamente conectados, siguiendo dos modalidades:
 - **Periódicamente:** Cada cierto intervalo de tiempo (del orden de varios segundos).
 - **Actualización gatillada:** Cuando la tabla de un nodo cambia, se envía inmediatamente una actualización a los vecinos.
- **Formato de la actualización:** Cada actualización consiste en una lista de pares:

(Destino, Costo)

- **Modificación de la tabla local:** La tabla de cada nodo se actualiza si se recibe una mejor ruta, determinada por:
 - **Costo menor:** La nueva ruta tiene un costo menor al registrado.
 - **Next-hop:** La ruta llegó desde un vecino inmediato que ofrece el camino más eficiente.
- **Mantenimiento de rutas:**
 - Las rutas existentes se refrescan con cada actualización recibida.
 - Si una ruta deja de recibir actualizaciones por un tiempo determinado, se elimina de la tabla (time out).

Este enfoque permite que cada nodo conozca las mejores rutas hacia todos los destinos utilizando únicamente información de sus vecinos inmediatos, lo que hace al protocolo escalable y distribuido.

2.2.1 Ejemplo I



(a) Subred. (b) Entrada de A, I, H, K y la nueva tabla de enrutamiento de J.

Contexto

El router J quiere calcular la mejor ruta hacia el destino G. Para ello:

- Recibe la información de sus vecinos inmediatos: A, I, H y K.
- Cada vecino le envía su vector de distancias a todos los nodos de la red.
- Costos directos de J a sus vecinos:
 - $J \rightarrow A$: 8 ms
 - $J \rightarrow I$: 10 ms
 - $J \rightarrow H$: 12 ms
 - $J \rightarrow K$: 6 ms

Cálculo de retardos hacia G

Usando la información de sus vecinos, J calcula el costo total hacia G de la siguiente forma:

$$\text{Costo } (J \rightarrow X \rightarrow G) = \text{Costo } (J \rightarrow X) + \text{Costo } (X \rightarrow G)$$

donde X es un vecino inmediato de J.

- A indica que llega a G con 18 ms. Entonces:

$$J \rightarrow A \rightarrow G = 8 + 18 = 26 \text{ ms}$$

- I indica que llega a G con 31 ms. Entonces:

$$J \rightarrow I \rightarrow G = 10 + 31 = 41 \text{ ms}$$

- H indica que llega a G con 6 ms. Entonces:

$$J \rightarrow H \rightarrow G = 12 + 6 = 18 \text{ ms}$$

- K indica que llega a G con 31 ms. Entonces:

$$J \rightarrow K \rightarrow G = 6 + 31 = 37 \text{ ms}$$

Selección de la mejor ruta

Comparando los valores:

$$26 \text{ (A)}, \quad 41 \text{ (I)}, \quad 18 \text{ (H)}, \quad 37 \text{ (K)}$$

El menor costo es 18 ms, a través de H.

Actualización de la tabla de ruteo de J

J registra en su tabla de enrutamiento:

- Destino: G
- Costo: 18 ms
- Next hop: H

2.2.2 Convergencia Global en Vector de Distancia

La **convergencia global** se refiere al momento en que todas las tablas de enrutamiento de todos los nodos de la red dejan de cambiar y reflejan de manera consistente las mejores rutas hacia todos los destinos.

1. Inicio:

- Cada nodo solo conoce la distancia a sus vecinos directos.
- Las rutas hacia destinos más lejanos no están definidas correctamente.

2. Intercambio de vectores:

- Los nodos envían periódicamente, o cuando hay cambios en su tabla, su vector de distancia a los vecinos.
- Cada nodo actualiza su tabla si recibe un costo menor hacia algún destino.

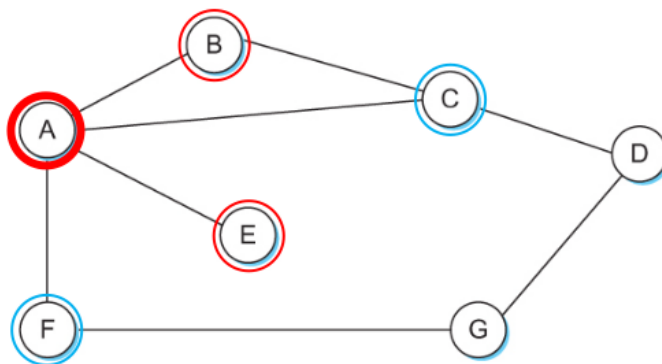
3. Propagación de información:

- Cada actualización puede desencadenar nuevas actualizaciones en otros nodos.
- Este proceso se repite hasta que todos los nodos tengan información consistente sobre los costos mínimos y los “next hops” hacia cada destino.

4. Convergencia global:

- Se alcanza cuando ninguna tabla cambia más.
- Todos los nodos coinciden sobre las rutas más cortas hacia todos los destinos.
- A partir de este momento, la red está estable y las decisiones de enrutamiento ya no generan oscilaciones.

2.2.3 Ejemplo II



En este ejemplo, cada nodo construye un vector que contiene la distancia (costo) a todos los demás nodos de la red y lo envía a sus vecinos inmediatos. Suponiendo que cada salto tiene un costo igual a 1, la tabla inicial del Nodo A es:

Destino	Costo	Next Hop
B	1	B
C	1	C
D	2	C
E	1	E
F	1	F
G	2	F

Estado inicial

Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	∞	1	1	∞
B	1	0	1	∞	∞	∞	∞
C	1	1	0	1	∞	∞	∞
D	∞	∞	1	0	∞	∞	1
E	1	∞	∞	∞	0	∞	∞
F	1	∞	∞	∞	∞	0	1
G	∞	∞	∞	1	∞	1	0

Destination	Cost	NextHop
B	1	B
C	1	C
D	∞	—
E	1	E
F	1	F
G	∞	—

Cada nodo mantiene únicamente su información local. Ningún nodo conoce inicialmente todo el estado de la red.

Convergencia global

Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	2	1	1	2
B	1	0	1	2	2	2	3
C	1	1	0	1	2	2	2
D	2	2	1	0	3	2	1
E	1	2	2	3	0	2	3
F	1	2	2	2	2	0	1
G	2	3	2	1	3	1	0

Destination	Cost	NextHop
B	1	B
C	1	C
D	2	C
E	1	E
F	1	F
G	2	F

Tabla final luego de una **convergencia global**

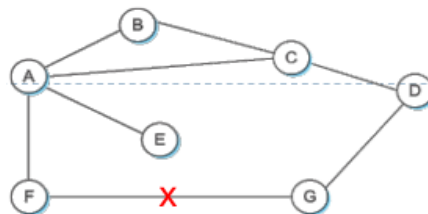
Tabla de ruteo final de A

Después de varias actualizaciones entre vecinos, cada nodo actualiza su tabla con la mejor ruta disponible hacia cada destino. La tabla de ruteo final de A refleja la convergencia global de la red.

2.2.4 Ciclos de Actualización

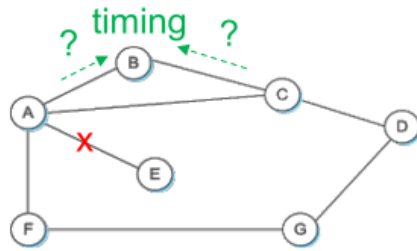
- **Caso feliz (estable):** Las rutas convergen rápidamente y los nodos mantienen información consistente.
- **Caso no feliz (inestable, conteo a infinito):** Surgen problemas cuando los enlaces fallan y las actualizaciones circulan creando rutas incorrectas temporalmente.

Ejemplo de estabilidad



- F detecta que su enlace a G falla. F fija distancia ∞ a G y envía actualización a A.
- A actualiza su distancia a G a ∞ porque usaba a F para llegar a G.
- A recibe luego una actualización de C indicando que G está a 2 saltos; A fija su distancia a 3 y envía actualización a F.
- F decide que puede llegar a G en 4 saltos vía A.

Ejemplo de inestabilidad



Importancia del Timing en Vector de Distancia

En el algoritmo de Vector de Distancia, las actualizaciones entre nodos no llegan todas al mismo tiempo. El **orden y el momento exacto** en que estas actualizaciones llegan y se procesan por los nodos se conoce como **timing**.

Supongamos que el enlace de A a E falla:

1. A detecta la falla y fija la distancia a E como ∞ , enviando actualización a B y C.
2. B y C todavía creen que pueden llegar a E por otras rutas y envían su información (distancia 2 a E) a A.
3. Debido al timing, A recibe esta información antes o después de haber actualizado correctamente su tabla, y recalcula su distancia a E, creyendo que puede llegar a E vía B o C en 3 o 4 saltos.
4. A envía esta actualización a C, que a su vez recalcula y comunica a B.
5. Este proceso puede repetirse varias veces, incrementando el “contador” de saltos, hasta que finalmente todos los nodos acuerdan que E es inalcanzable.

El Timing

- Determina la velocidad y el orden de propagación de las rutas entre nodos.
- Es la razón por la que un enlace caído puede causar oscilaciones temporales en las rutas.
- Explica el fenómeno del **conteo a infinito**, donde las distancias incorrectas se van incrementando hasta que la red finalmente converge.

Problema de Conteo a Infinito

En los protocolos de Vector de Distancia, cuando un enlace falla o un nodo se desconecta, los routers actualizan sus tablas enviando distancias mayores hacia ese destino.

El **conteo a infinito** ocurre cuando:

- La información de fallo se propaga lentamente por la red.
- Los nodos calculan rutas “alternativas” utilizando información desactualizada de sus vecinos.
- Como resultado, los nodos incrementan progresivamente el costo hacia ese destino sin llegar a un valor estable de inmediato.

En otras palabras, los nodos “se pasan la pelota” aumentando el número de saltos en cada actualización hasta alcanzar el infinito.

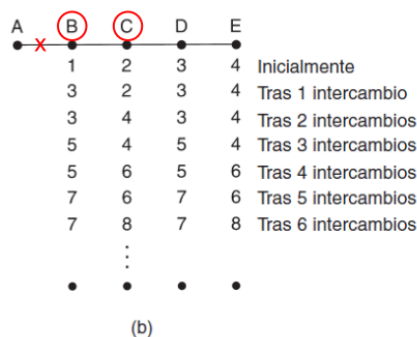
Caso (a): Nodo A está seteado como infinito y luego se activa

A	B	C	D	E	
•	•	•	•	•	Inicialmente
	1	•	•	•	Tras 1 intercambio
	1	2	•	•	Tras 2 intercambios
	1	2	3	•	Tras 3 intercambios
	1	2	3	4	Tras 4 intercambios

(a)

- Inicialmente, A indica que no puede alcanzar ciertos destinos (distancia = ∞).
- Cuando A se activa de nuevo, comienza a enviar información de rutas válidas.
- La red debe recalcular las rutas hacia A y sus destinos asociados.
- El conteo a infinito puede ocurrir si algunos vecinos todavía tenían rutas viejas que involucraban a A, porque reciben primero las rutas antiguas antes de actualizarse con la información de que A está activo.

Caso (b): Nodo A está activo y luego cae



- A estaba activo y comunicando rutas normales.
- De repente A falla o su enlace cae.
- Los vecinos reciben primero la información de que A ya no puede alcanzar ciertos destinos.
- Sin embargo, algunos vecinos todavía creen que A está disponible y reportan rutas que pasan por A.
- Esto genera oscilaciones temporales y aumento progresivo del costo hacia esos destinos, típico del conteo a infinito.

2.2.5 Heurísticas para Romper Ciclos en Vector de Distancia

En los protocolos de Vector de Distancia, los ciclos en la red pueden provocar que la información de rutas incorrecta se propague indefinidamente, generando el problema de **conteo a infinito**. Para mitigar esto, se utilizan varias heurísticas:

1. Fijar un valor máximo como infinito

- Se establece un costo máximo (por ejemplo 16) que representa un destino inalcanzable.
- Cuando el costo de una ruta alcanza este valor, el nodo asume que no hay camino hacia ese destino.
- Esto rompe el ciclo porque detiene el incremento indefinido de la distancia.

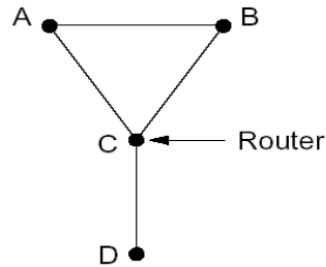
2. Split Horizon

- Un nodo no envía información sobre rutas a un vecino si esas rutas fueron aprendidas de ese mismo vecino.
- Esto evita ciclos directos entre dos nodos, que podrían retroalimentarse información incorrecta.
- Limitación: solo previene ciclos entre dos nodos; ciclos más largos no se resuelven.

3. Split Horizon with Poison Reverse

- Variante del Split Horizon.
- El nodo sí envía la ruta al vecino de donde la aprendió, pero marca el costo como ∞ .
- El vecino sabe que la ruta existe, pero no la usará para calcular su propio camino.
- También solo evita ciclos de dos nodos.

Ejemplo



- Paso (a): Todos los costos son uno
 - Cada nodo conoce sus vecinos directos.
 - Las tablas de enrutamiento reflejan los costos mínimos hacia todos los destinos cercanos.

Nodo	Destino	Costo / NextHop
A	D	2 / B
B	D	1 / D
C	D	1 / D

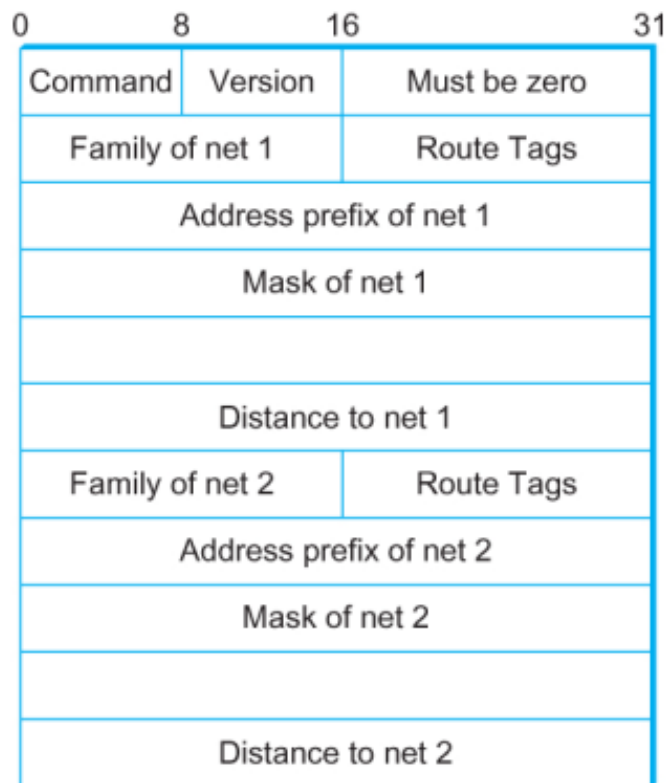
No hay ciclos ni inconsistencias en este estado.

- Paso (b): Falla el enlace C – D
 - C detecta que ya no puede llegar a D directamente.
 - Con **Poison Reverse**, C envía a sus vecinos la información de D con costo ∞ .
 - Esto evita que se genere un ciclo de dos nodos (conteo a infinito) entre C y sus vecinos.
- Paso (c): A puede llegar a D por B
 - A recibe la actualización de C indicando costo ∞ hacia D.
 - A revisa otras rutas y determina que puede alcanzar D vía B.
 - Tabla de A actualizada:

Destino D, Costo 2, NextHop B

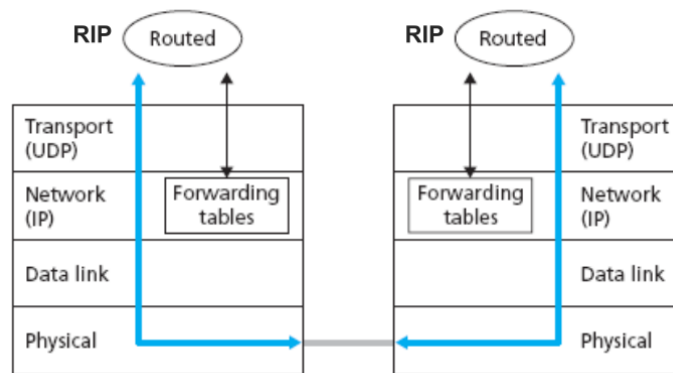
- Gracias a Poison Reverse, C no utiliza información de A sobre D para recalcular su propia ruta hacia D, evitando ciclos.
- Paso (d): Convergencia
 - El proceso de actualización continúa hasta que todas las tablas de la red se estabilizan.
 - Los costos hacia D se fijan en los valores correctos, evitando el conteo a infinito.
 - Poison Reverse garantiza que las rutas aprendidas de un vecino no se utilicen para retroalimentarlo, rompiendo ciclos directos de dos nodos.

2.3 Formato paquete RIPv2



- **Command (1 byte):** Indica el tipo de mensaje: *Request* (solicitud de rutas) o *Response* (respuesta con rutas).
- **Version (1 byte):** Versión del protocolo, para RIPv2 este valor es 2.
- **Unused / Reserved (2 bytes):** Reservado para futuros usos, generalmente se establece en 0.
- **Route Entries (cada una 20 bytes):** Contiene información sobre cada ruta, incluyendo:
 - **Address Family Identifier (AFI) (2 bytes):** Identifica el tipo de dirección contenida en la entrada. Para IPv4 el valor es 2.
 - **Route Tag (2 bytes):** Campo opcional usado para distinguir rutas externas de internas o para etiquetar rutas de forma administrativa.
 - **IP Address (4 bytes):** Dirección IPv4 del destino de la ruta.
 - **Subnet Mask (4 bytes):** Máscara de subred asociada a la dirección de destino, utilizada para definir la red.
 - **Next Hop (4 bytes):** Dirección IPv4 del siguiente salto a usar para alcanzar el destino. Si se pone 0.0.0.0, se asume que se usa directamente la dirección del origen del paquete RIP.
 - **Metric (4 bytes):** Costo de la ruta hacia el destino. Valor entre 1 y 16, donde 16 indica que el destino es inalcanzable.

2.4 RIP como un "daemon"



- **RIP como daemon:**

- En sistemas Unix/Linux, un **daemon** es un programa que se ejecuta en segundo plano, sin intervención directa del usuario.
- RIP se implementa como un daemon que corre continuamente, actualizando automáticamente las tablas de enrutamiento.
- El daemon se encarga de:
 - * Enviar periódicamente vectores de distancia a los vecinos.
 - * Recibir vectores de distancia de los vecinos.
 - * Actualizar la tabla de ruteo según las reglas de RIP (mejor ruta, split horizon, poison reverse, etc.).

- **Uso de UDP:**

- RIP utiliza **UDP (User Datagram Protocol)** como protocolo de transporte.
- UDP es no orientado a conexión, lo que significa que:
 - * No garantiza la entrega de paquetes.
 - * No requiere establecer una conexión antes de enviar datos.
 - * Es rápido y liviano, ideal para enviar tablas de enrutamiento periódicas.

- **Puerto reservado:**

- RIP utiliza el puerto UDP **520** de forma estándar.
- Todos los routers que ejecutan RIP escuchan en este puerto.
- Las actualizaciones de RIP se envían y reciben usando este puerto, asegurando comunicación correcta entre routers.

2.5 Algoritmo de Estado del Enlace - Link State

El **Algoritmo de Estado del Enlace** o *Link State* es un enfoque de enrutamiento donde todos los nodos de la red poseen información completa y consistente sobre la topología de la red y los costos de los enlaces. Este enfoque contrasta con los algoritmos de Vector de Distancia, donde cada nodo solo conoce información de sus vecinos inmediatos.

2.5.1 Principios Fundamentales

- **Link State Broadcast:** Cada nodo mantiene la misma información sobre la topología de la red y los costos asociados a cada enlace.
- **Cómputo de caminos mínimos:** Una vez que la topología completa está disponible, cada nodo utiliza el **algoritmo de Dijkstra** (forward search) para calcular las rutas más cortas hacia todos los destinos.

2.5.2 Pasos del Algoritmo de Estado de Enlace

1. **Descubrimiento de vecinos:** Cada nodo identifica sus vecinos directamente conectados y conoce sus direcciones de red.
2. **Medición de costos:** Cada nodo mide el costo de los enlaces hacia sus vecinos.
3. **Construcción del paquete de estado:** El nodo crea un *Link State Packet (LSP)* que contiene toda la información que acaba de aprender sobre sus enlaces.
4. **Difusión del paquete:** El nodo envía su LSP a todos los demás routers de la red.
5. **Cálculo de rutas:** Una vez recibida la información de todos los LSP, cada nodo calcula las rutas más cortas hacia todos los demás nodos usando el algoritmo de Dijkstra.

2.5.3 Paquete de Estado del Enlace (LSP)

Cada *Link State Packet* contiene:

- **ID del router:** Identificador del nodo que generó el LSP.
- **Costo de enlaces:** Para cada vecino directamente conectado.
- **Número de secuencia (SEQNO):** Permite identificar actualizaciones recientes y descartar duplicados.
- **Time-To-Live (TTL):** Limita la vida del paquete para evitar inundación infinita.

2.5.4 Inundación Confiable (Reliable Flooding)

Cada nodo envía información únicamente sobre sus enlaces directamente conectados, no sobre toda la tabla de enrutamiento. Esta información se **inunda** por toda la red, asegurando que todos los nodos tengan la misma visión de la topología. Cada router maneja los *Link State Packets (LSP)* de forma confiable para asegurar que toda la red tenga la misma información:

- **Almacenamiento:** Cada router guarda el LSP más reciente de cada nodo.
- **Decremento de TTL:** El *Time-To-Live* de cada LSP se decrementa periódicamente. Si $TTL = 0$, el LSP se descarta.
- **Reenvío (Flooding):** Cada router reenvía el LSP a todos sus vecinos **excepto** al que se lo envió originalmente.
- **LSP periódicos:** Cada router genera periódicamente un nuevo LSP.
- **Número de secuencia (SEQNO):** Incrementa en cada LSP generado para identificar actualizaciones nuevas. Al reiniciar el router, SEQNO comienza en 0.

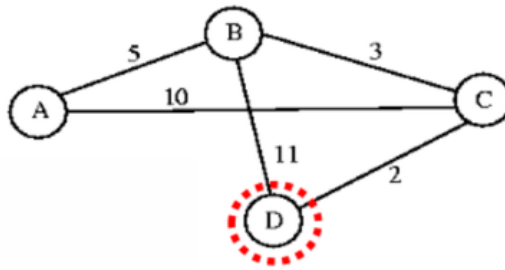
2.5.5 Cálculo de la Ruta Más Corta (Shortest Path Routing)

- El cálculo de la tabla de ruteo se realiza **conforme llegan los LSP**.
- Se utiliza una implementación del algoritmo de Dijkstra conocida como **forward search algorithm**.
- Se manejan dos listas de entradas:
 - **Lista tentativas:** nodos que se van a explorar.
 - **Lista confirmadas:** nodos cuya ruta más corta ya se determinó.
- Cada entrada en estas listas contiene: (*Destination*, *Cost*, *NextHop*)

2.5.6 Uso del Algoritmo de Dijkstra en Ruteo

El algoritmo de Dijkstra se utiliza en protocolos de **Estado de Enlace** para calcular la **ruta más corta** desde un nodo origen hacia todos los demás nodos de la red. Este proceso permite construir la tabla de ruteo óptima para cada nodo.

Construcción de la Tabla de Ruteo para el nodo D



Se consideran dos conjuntos de nodos:

- **Confirmados:** nodos cuya ruta más corta desde D ya se ha determinado.
- **Tentativos:** nodos aún no confirmados, pero con un costo calculado provisionalmente.

El procedimiento paso a paso para construir la tabla de ruteo hacia todos los destinos:

1. Inicialmente, se examinan los vecinos directos de D y se incorporan a la lista tentativa con su costo directo.
2. Se selecciona el nodo con **menor costo** entre los tentativos.
 - Ejemplo: C es el menor costo.
 - Se examina el LSP del nodo recién confirmado para actualizar costos.
3. Se incorporan rutas hacia otros nodos usando el nodo confirmado:
 - Con C, se actualizan los costos y se incorpora A vía C.
4. Se mueve nuevamente el nodo de menor costo de la lista tentativa a la lista confirmada.
 - Ejemplo: B es confirmado y se usan sus LSP para actualizar costos hacia otros nodos.
5. Se repite el proceso hasta que todos los nodos estén confirmados.
 - Finalmente, A es el único nodo restante y se confirma.

Cada entrada de la tabla de ruteo tiene el formato: (Destination, Cost, NextHop)

Paso	Confirmado	Tentativo
1	(D,0,-)	
2	(D,0,-)	(B,11,B) (C,2,C)
3	(D,0,-) (C,2,C)	<u>(B,11,B)</u> ↓
4	(D,0,-) (C,2,C)	<u>(B,5,C)</u> (A,12,C)
5	(D,0,-) (C,2,C) (B,5,C)	<u>(A,12,C)</u> ↓
6	(D,0,-) (C,2,C) (B,5,C)	<u>(A,10,C)</u>
7	(D,0,-) (C,2,C) (B,5,C) (A,10,C)	

Pseudocódigo del Algoritmo de Dijkstra para ruteo

Sea:

- N : conjunto de nodos del grafo.
- $l(i, j)$: costo no negativo del arco (i, j) . Si no hay arco, $l(i, j) = \infty$.
- s : nodo origen (self).
- M : conjunto de nodos incorporados hasta el momento.
- $C(n)$: costo mínimo conocido desde s hasta el nodo n .

```
1 M = {s}
2 for each n in N - {s}:
3     C(n) = l(s, n)
4
5 while (N != M):
6     seleccionar w en (N - M) tal que C(w) sea mínimo
7     M = M union {w}
8     for each n in (N - M):
9         C(n) = MIN(C(n), C(w) + l(w, n))
```

2.6 Comparación entre Algoritmos de Ruteo: Vector de Distancia vs Estado de Enlaces

Vector de Distancia (Distance Vector)

- Cada nodo transmite a sus vecinos toda la información que posee sobre la red, es decir, la distancia a todos los nodos.
- Puede ser inestable en ciertas condiciones, generando problemas como **conteo a infinito**.
- La convergencia depende de la propagación de las actualizaciones entre los vecinos, lo que puede tardar varios ciclos.

Estado de Enlaces (Link State)

- Cada nodo transmite a toda la red información únicamente sobre el estado de sus vecinos directos.
- Es un algoritmo más estable, que no genera excesivo tráfico y responde rápidamente a cambios de topología.
- Requiere que cada nodo almacene un **Link State Packet (LSP)** por cada nodo de la red, lo que implica mayor uso de memoria.

Resumen Comparativo

- **Vector de Distancia:** sencillo, pero susceptible a inestabilidad y conteo a infinito.
- **Estado de Enlaces:** estable y eficiente ante cambios, aunque requiere más almacenamiento y procesamiento.

2.7 Métricas de Enlace en ARPANET

Métrica original

- Medía el número de paquetes encolados en cada enlace.
- No consideraba la latencia ni el ancho de banda del enlace.

Métrica revisada

- Cada paquete se marca con su **tiempo de llegada** (*Arrival Time, AT*) y se graba el **tiempo de salida** (*Departure Time, DT*).
- Cuando llega el ACK del enlace, se calcula el retardo:

$$\text{Delay} = (DT - AT) + \text{Transmit} + \text{Latency}$$

donde *Transmit* y *Latency* son parámetros estáticos del enlace.

- En caso de timeout, se reinicia DT al tiempo de salida para retransmisión.
- El costo del enlace se calcula como el **retardo promedio** sobre un período determinado.

Mejoras adicionales

- Reducción del *rango dinámico* para el costo, moderando la diferencia entre el peor y mejor enlace.
- En lugar del retardo, se puede emplear la **utilización del enlace** como métrica para reflejar la carga real de la red.

2.8 OSPF (Open Shortest Path First)

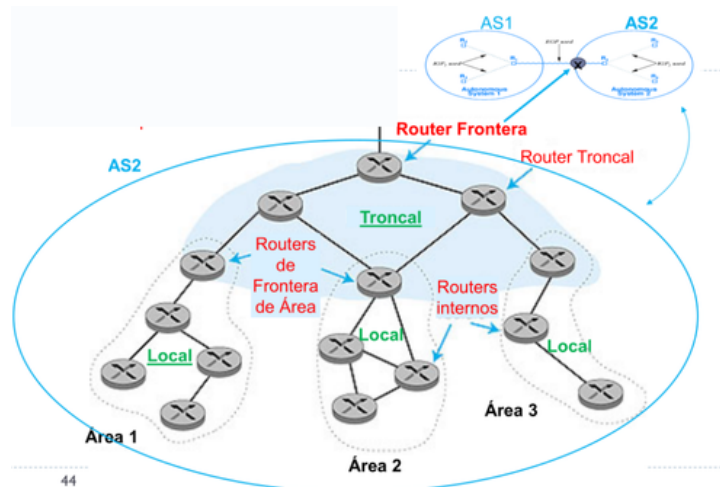
OSPF (Open Shortest Path First) es un protocolo de enrutamiento interior (IGP) utilizado en redes de área local (LAN) y redes más grandes dentro de un Autonomous System (AS). Su objetivo principal es permitir que los routers conozcan la topología de la red y calculen las rutas más cortas hacia todos los destinos.

- **Open:** código abierto, disponible públicamente.
- OSPF utiliza un **algoritmo de Estado de Enlaces** para el enrutamiento.
- Cada nodo mantiene un mapa de la topología de la red y calcula rutas usando el **algoritmo de Dijkstra**.
- Los anuncios OSPF contienen información de cada router vecino.
- Los anuncios se distribuyen a todos los nodos dentro de un **Autonomous System (AS)** mediante inundación.
- Los mensajes OSPF se transportan directamente sobre IP, sin usar TCP o UDP.

2.8.1 Características Principales

- **Seguridad:** todos los mensajes OSPF están autenticados para prevenir intrusiones maliciosas.
- **Multipath:** permite múltiples rutas con el mismo costo hacia un destino, a diferencia de RIP que solo permite un camino.
- **Múltiples métricas de enlace:** diferentes costos por tipo de servicio (*Type of Service, ToS*), permitiendo optimizar rendimiento según el tráfico.
- **Soporte uni- y multicast:** la multidifusión OSPF (MOSPF) usa la misma base de datos topológica que OSPF normal.

2.8.2 OSPF Jerárquico



- Para grandes dominios, OSPF puede organizarse jerárquicamente en áreas.
- **Dos niveles de jerarquía:**
 - **Área local:** cada nodo conoce la topología interna del área.
 - **Área troncal:** conecta varias áreas y permite el intercambio de información resumida.
- **Routers de frontera:** conectan con otros AS y resumen distancias a redes del área.
- **Routers troncales:** ejecutan el ruteo limitado al área troncal.
- Cada nodo solo conoce la ruta más corta hacia redes de otras áreas; no toda la topología de otras áreas.

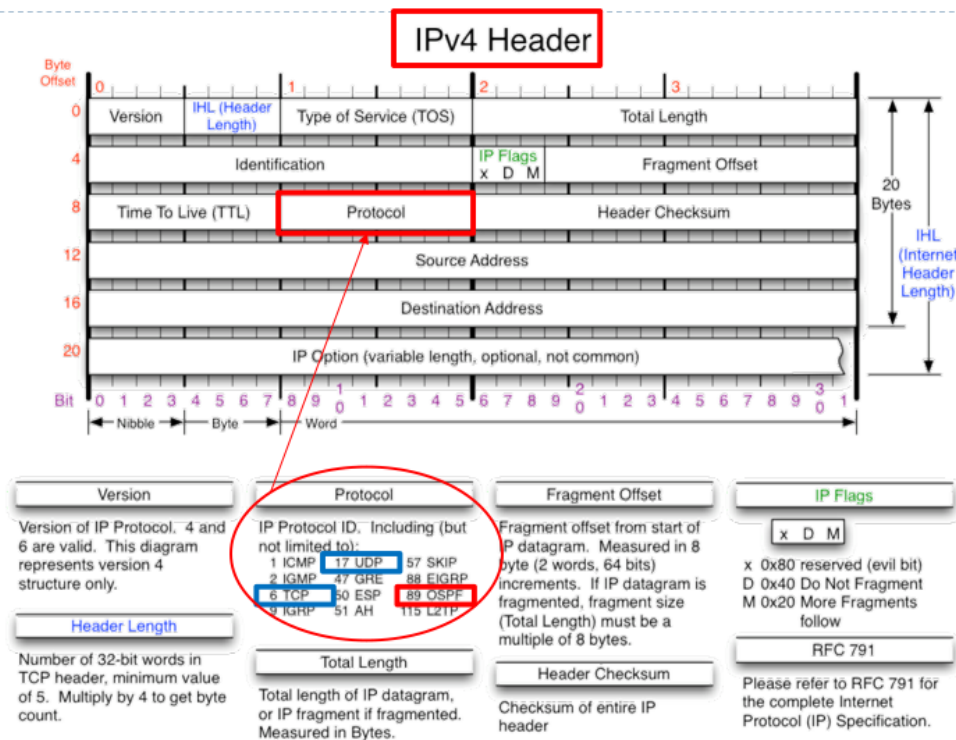
2.8.3 Tipos de Mensajes en OSPF

- **Link State Update (LSU):** inunda periódicamente la información del estado de enlaces a routers adyacentes. Contiene el costo de los enlaces y actualiza la base de datos topológica.
- **Database Description (DBD):** intercambia los números de secuencia de las entradas del estado de enlace para identificar la información más reciente.
- **Link State Request (LSR):** solicita información específica de estado de enlace a un vecino.
- **Confirmaciones (ACK):** todos los mensajes DBD, LSR y LSU se confirman para asegurar confiabilidad.

2.8.4 Mecanismo de Convergencia

- Cada router inunda periódicamente los mensajes **LINK STATE UPDATE** a sus vecinos para comunicar su estado y los costos de sus enlaces.
- Cada mensaje lleva un **número de secuencia** para identificar si es más reciente que la información que ya posee el receptor.
- Cuando una línea cambia de costo, los routers generan nuevos mensajes LSU para propagar la actualización.
- Los mensajes DBD permiten comparar la información de cada vecino y determinar qué datos están desactualizados.
- Cualquier router puede solicitar información específica de estado de enlace mediante LSR.
- El resultado es que cada par de routers adyacentes verifica qué datos son más recientes y difunde las actualizaciones a lo largo del área, garantizando que todos los nodos converjan con la misma base de datos topológica.

2.8.5 Modificación del IPv4 Header al usar OSPF



Cuando un router envía un paquete OSPF, este se transporta **directamente sobre IPv4**, sin TCP ni UDP. El header IPv4 se adapta de la siguiente manera:

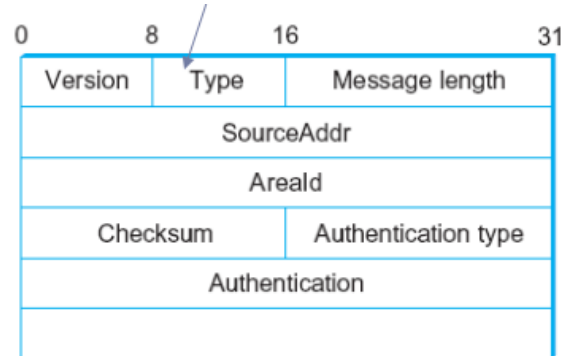
1. **Protocolo IPv4:** En el campo **Protocol** del header IPv4 se indica 89, que corresponde a OSPF. Esto permite que el router receptor interprete correctamente el payload.
2. **Direcciones IP:**
 - **Source IP:** dirección del router emisor.

- **Destination IP:** depende del tipo de mensaje OSPF:

- Mensajes **Hello** dentro de un área: multicast 224.0.0.5 (todos los routers OSPF).
- Mensajes de **Routing Update / LSA:** multicast 224.0.0.6 (routers designados) o unicast a un router específico.

3. **TTL (Time to Live):** OSPF generalmente usa TTL = 1 para mensajes dentro del área local, evitando que salgan de la red local.
4. **Longitud total:** El campo Total Length se ajusta para incluir el tamaño del header IPv4 más el paquete OSPF.
5. **Checksum del header IPv4:** Se recalcula automáticamente debido a cambios en la dirección destino, TTL o longitud.
6. **Opciones IPv4:** OSPF típicamente utiliza un header estándar de 20 bytes y no requiere modificar opciones especiales.

2.8.6 Formato de Header OSPF



El header de OSPF se encapsula directamente en un paquete IP y contiene campos esenciales para la operación del protocolo:

- **Version:** versión de OSPF (por ejemplo, 2).
- **Type:** tipo de mensaje OSPF (Hello, Database Description, Link State Request, Link State Update, Link State Acknowledgment).
- **Packet Length:** longitud total del paquete OSPF.
- **Router ID:** identificador único del router emisor.
- **Area ID:** identifica el área OSPF a la que pertenece el router.
- **Checksum:** verificación de integridad del paquete OSPF.
- **Authentication:** campo opcional para autenticación de mensajes.

2.8.7 OSPF Link State Advertisement (LSA)

LS Age		Options	Type = 1
Link-state ID			
Advertising router			
LS sequence number			
LS checksum		Length	
0	Flags	0	Number of links
Link ID			
Link data			
Link type	Num_TOS	Metric	
Optional TOS information			
More links			

Las **LSAs** son la unidad de información de topología en OSPF y contienen:

- **LS Age:** tiempo que ha estado activo el LSA.

- **LS Type:** tipo de LSA (Router LSA, Network LSA, Summary LSA, AS-external LSA, etc.).
- **Link State ID:** identifica de manera única el objeto que describe el LSA.
- **Advertising Router:** router que generó el LSA.
- **LS Sequence Number:** número de secuencia para versiones de LSA.
- **LS Checksum:** verificación de integridad del contenido del LSA.
- **Length:** longitud total del LSA.
- **Link Information:** detalles de los enlaces del router, costos, direcciones de vecinos y redes conectadas.

3 Ruteo Interdominio

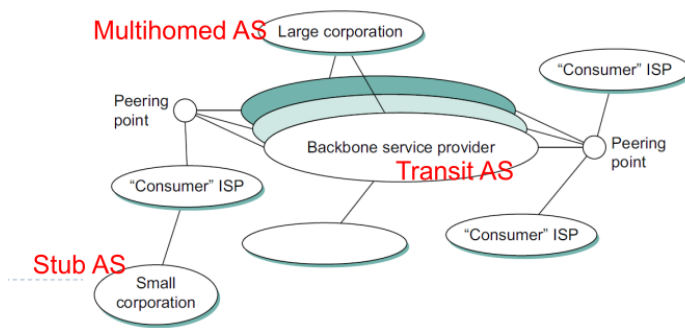
3.1 EGP: Exterior Gateway Protocol (antecesor)

- Diseñado para Internet organizada como un árbol.
- Se enfoca en la alcanzabilidad de nodos, no en optimizar rutas.
- Mensajes y funcionamiento:
 - **Adquisición de vecinos:** un router requiere que otro sea su par y se intercambia información de alcance.
 - **Alcance de vecinos:** los routers prueban periódicamente si el vecino sigue alcanzable, usando mensajes HELLO/ACK.
 - **Actualización de rutas:** los pares intercambian periódicamente sus tablas de ruteo (vector de distancia).

3.2 BGP: Exterior Gateway Protocol Moderno

- Internet está organizada en **Sistemas Autónomos**, cada uno bajo control administrativo único.
- BGP se encarga del **ruteo interdominio**, resolviendo cómo alcanzar redes en otros AS sin revelar la topología interna.
- Cada AS puede ejecutar su propio ruteo intradominio mientras BGP se ocupa de la interconexión entre AS.

3.2.1 Tipos de Sistemas Autónomos en BGP

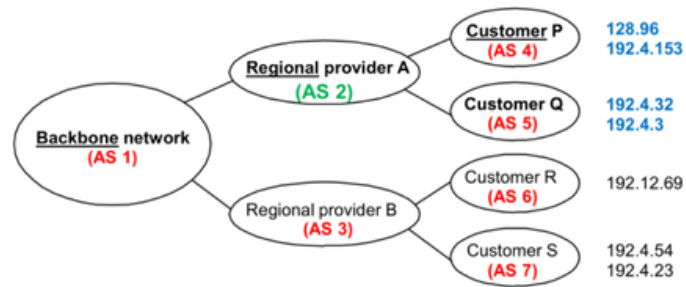


- **Stub AS:** única conexión a otro AS; solo transporta tráfico local.
- **Multihomed AS:** conexiones a más de un AS; no transporta tráfico en tránsito.
- **Transit AS:** conexiones a más de un AS; transporta tráfico local y en tránsito.

3.2.2 Routers y Portavoces BGP

- Cada AS tiene uno o más **routers de borde**.
- Cada AS designa al menos un **portavoz BGP** que publica:
 - Redes locales del AS.
 - Redes alcanzables a través del AS (en caso de Transit AS).
 - Información de rutas (paths AS) para interconectar con otros AS.

Ejemplo de BGP



- El portavoz de AS2 publica alcanzabilidad a las redes de sus clientes P y Q:
 - Red 128.96
 - Red 192.4.153
 - Red 192.4.32
 - Red 192.4.3
- El portavoz del backbone (AS1) anuncia que estas redes son alcanzables a través de la ruta (AS1, AS2).
- Un portavoz puede retirar una ruta previamente anunciada.

3.3 Network Access Points (NAPs)

- Los **NAPs** o **Internet eXchanges (IXs)** son puntos de interconexión entre redes de distintos operadores y entidades.
- Objetivos principales:
 - Intercambiar tráfico de manera eficiente entre diferentes redes.
 - Mejorar la calidad de servicio.
 - Reducir costos de interconexión.
- Los NAPs CABASE siguen un modelo cooperativo, donde todos los miembros buscan mejorar la calidad de las comunicaciones y minimizar costos.